Modular Design of a Software System

Applied to the `kvstore` Distributed Key–Value Store

Software Engineering — Tutorial & Lab 8 Deliverable

MCA — Second Semester

High cohesion

Low coupling

Clear interfaces

Project: `kvstore` — a distributed, transactional key–value store
Written in: Go (standard library only, zero dependencies)
Submitted by: Aditi Garg
Date: October 15, 2026

This document is the requested lab-8 artifact: it divides a real project into modules, justifies cohesion and coupling decisions, documents interfaces, and solves the tutorial numericals on the resulting module graph.

1 Aim and Learning Outcomes

Aim. To design the modular structure of the own project `kvstore`: identify its main responsibilities, group them into modules with one clear job each, draw the resulting module diagram, and state the interfaces through which the modules communicate — while keeping *high cohesion* and *low coupling*.

After this exercise, we can:

- identify the main functions of a real software project and express them as short responsibilities;
- divide those functions into suitable modules with meaningful names;
- explain high cohesion and low coupling in easy words, and justify concrete design decisions;
- draw a labeled module diagram and describe each module interface with inputs and outputs.

2 Background in Easy Words

What is a module?

A **module** is a small part of a large software system that has *one clear main job*. It is a unit of understanding, building, testing and changing (in `kvstore`, each Go package is a module). Modules work together only through *defined interfaces*.

Why divide a project into modules?

1. It makes a big project easier to understand: one file tree per idea.
2. Different team members can work on different parts at the same time.
3. It simplifies testing, maintenance and future changes — modules such as `internal/ai` are tested completely alone, with no cluster running.
4. One error is easier to locate: a wrong timestamp points to the clock module, a wrong leader election points to the consensus module.

Cohesion and coupling

Cohesion = how closely the tasks inside one module belong together. High cohesion means "one module, one main purpose".
Coupling = how much one module depends on other modules. Low coupling means connections are necessary, limited and clear (ideally through an interface, never by touching another module's private data).

3 The Project Under Design: `kvstore`

`kvstore` is a distributed, transactional key–value store written in Go. The data model is simple (keys are strings, values are byte sequences, and several operations can be grouped into one all-or-nothing transaction), but the system around it is substantial because the data must survive crashes, network partitions and clock drift. Its requirements — the starting point for all further steps — are:

#	Requirement	What it means in one line
R1	**Key–value operations**	Put, Get, Delete, and atomic multi-key transactions.
R2	**Fault-tolerant replication**	Data is replicated across machines with the Raft consensus algorithm; the cluster keeps working when a minority of nodes fails.
R3	**Strong isolation**	Transactions get snapshot isolation; conflicting writes are resolved by first-committer-wins, checked identically on every replica.
R4	**Durability**	Every accepted change is written to a checksummed write-ahead log and fsynced *before* being acknowledged.
R5	**Deterministic ordering**	Timestamps come from a hybrid logical clock, giving a total order that no two nodes can ever reverse.
R6	**Observability**	Counters, gauges and histograms exported in Prometheus and JSON formats, plus a REST API and a live browser dashboard.
R7	**Operational intelligence**	Transparent statistical analysis: anomaly detection, correlation, trend forecasting, leadership advice, auto-tuning and natural-language queries.
R8	**Horizontal scaling**	When one cluster is not enough, independent clusters (shards) sit behind one gateway address, with cross-shard two-phase-commit transactions.

Table 1: Requirement list (5–8 important requirements of the own project).

A deliberate project constraint: `kvstore` uses *no third-party libraries at all* — Raft, MVCC, the WAL, the wire protocol and the HTTP layer are all implemented with only the Go standard library. This is possible *because* the system is modular: each layer is small enough to be fully understood and tested on its own.

4 Step 1 — List the Responsibilities

Everything the system must do, written as one short action each:

1. store a value under a key
2. read the value of a key
3. delete a key
4. run several operations as one atomic transaction
5. detect write-write conflicts
6. replicate every operation to other nodes
7. elect a leader and recover after leader failure
8. keep the log safe on disk before acknowledging
9. generate ordered timestamps across nodes
10. recover state after a crash (log replay, snapshots)
11. serve requests over the network between nodes
12. redirect clients that reach a non-leader node
13. measure latency, throughput and failures
14. export metrics to Prometheus and JSON
15. show a live dashboard with cluster state
16. detect anomalies in observed behavior
17. correlate and forecast metric trends
18. advise leadership transfers of unhealthy leaders
19. auto-tune GC and snapshot parameters
20. answer plain-English questions about the cluster
21. route keys to the correct shard
22. run a transaction across several shards
23. discover the current leader and retry there
24. encrypt connections (optional TLS / mutual TLS)
25. inject faults (partitions, kills, delays) in tests

5 Step 2 — Group the Responsibilities into Modules

Related actions are put together; unrelated ones are kept apart. Following the tutorial example (add book + edit book + search book → **Book Catalogue**), we apply the same rule here, e.g. “append + checksum + fsync + replay” → **Durable Log**, and we keep login-level concerns (security helpers) separate from data-plane ones.

1. store / read / delete → part of the **Transaction Engine**
2. transactions + conflict detection → **Transaction Engine**
3. replication + election + recovery → **Consensus**
4. append + checksum + fsync + replay → **Durable Log**
5. timestamps → **Hybrid Clock**
6. moving bytes between nodes → **Wire Protocol**
7. leader discovery + retry → **Client Library**
8. measuring + exporting metrics → **Metrics Registry**
9. anomaly / correlation / forecast / advice → **Operational AI**
10. REST API + dashboard → **Web Layer**
11. key-to-shard routing → **Routing Ring**
12. certificates → **Security Helpers**
13. simulated network failures → **Fault Injection**

Naming rule

Each module gets a short, meaningful name and — since this is a real codebase — its exact Go package, so the design can be checked against the code.

Module	Package	One main responsibility
CLI Client	`cmd/kvctl`	the command-line user talks to the cluster
Web Layer	`internal/httpapi`	REST API, Server-Sent-Events stream, embedded browser dashboard
Sharding Server	`cmd/kvgateway`	one logical address in front of several independent clusters
Transaction Engine	`internal/storage`	the Raft state machine: Begin / Get / Put / Delete / Commit transactions
Operational AI	`internal/ai`	analyze metrics: anomalies, correlations, forecasts, advice, NL queries
Routing Ring	`internal/shard`	decide, for every key, which shard is responsible (rendezvous hashing)
Consensus	`internal/raft`	agree on one ordered operation log: elections, replication, snapshots, membership
Version Store	`internal/mvcc`	keep version chains per key; snapshot isolation and conflict checks
Hybrid Clock	`internal/hlc`	globally unique, causally ordered timestamps
Write-Ahead Log	`internal/wal`	append arbitrary records safely and replay them faithfully after a crash
Wire Protocol	`internal/transport`	encode a message envelope, send it over TCP, bring the reply back (optional TLS)
Client Library	`internal/client`	find the leader, follow NotLeader hints, retry across nodes
Metrics Registry	`internal/metrics`	counters, gauges, histograms; Prometheus & JSON export
Supporting (not on the main diagram): Security Helpers `internal/tlsutil` (certificate generation/loading); Fault Injection `internal/fault` (test-only simulated network).		

Table 2: Module list: module name, package, and the—single—main responsibility.

6 Module Diagram

The system is deliberately organized in **layers**: an arrow $A \to B$ means "module A depends on module B". Upper layers depend on lower layers; lower layers never import upper ones, which keeps every layer independently testable.

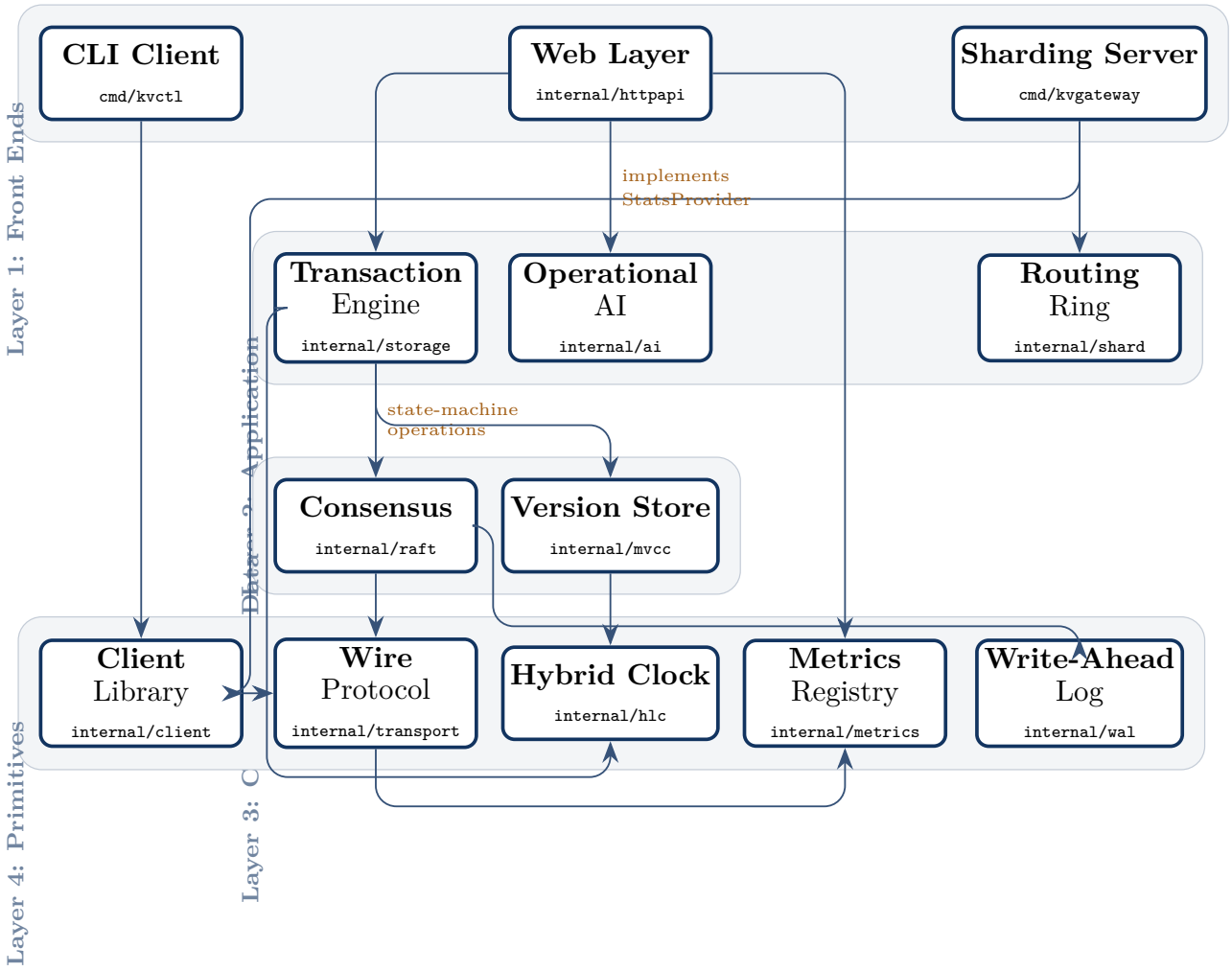


Figure 1: Module diagram of `kvstore`. Boxes are modules; arrows are import dependencies (direction of dependence). Numbers (1)–(14) correspond to the connection list in Section 10. The Fault-Injection module (`internal/fault`) exercises the Wire Protocol in tests only; its arrow is omitted for clarity to keep the production picture clean.

How to read the diagram

- **Layer 4** are primitives with no secrets: the clock, the disk log, the wire format, the metrics counters. They change rarely.
- **Layer 3** holds the two hardest ideas: consensus and versioned storage.
- **Layer 2** combines them into a transactional database and adds intelligence.
- **Layer 1** is what users actually touch; it may know everything below it — a composition root is allowed to depend widely, *locally*.

7 Step 5 — Module Interfaces

An **interface** is a defined way for two modules to communicate: it fixes the inputs, the outputs and the modules involved, so each side can be built, tested and changed independently. Four interfaces of `kvstore` are documented here (the lab asks for three; a fourth makes the design complete).

Interface 1: Transaction API (Storage Engine ↔ Web Layer)

```
// internal/storage/engine.go
func (e *Engine) Begin(ctx context.Context) (*Txn, error)
```

```
func (t *Txn)    Get(key string) (value []byte, ok bool)
func (t *Txn)    Put(key string, value []byte)
func (t *Txn)    Delete(key string)
func (t *Txn)    Rollback()
func (t *Txn)    Commit(ctx context.Context) (hlc.Timestamp, error) // first-committer-
    wins check
```

Call	Input	Output	Modules involved
Begin	context	transaction handle (with snapshot timestamp)	Web Layer → Transaction Engine
Get	`key`	value bytes, or "not found"	Web Layer → Transaction Engine
Put/Delete	`key`, value	changes buffered in the transaction	Web Layer → Transaction Engine
Commit	open transaction	commit timestamp, or abort on write conflict	Transaction Engine → Consensus, Version Store
Rollback	open transaction	nothing (transaction discarded)	Web Layer → Transaction Engine

Table 3: Transaction API — one defined way to read and write the database.

Interface 2: Wire Envelope (Wire Protocol ↔ everyone below)

```
// internal/transport/protocol.go
type Envelope struct {
    Kind            byte        // RequestVote | AppendEntries | InstallSnapshot |
                                // TimeoutNow   | ClientRequest
    RequestVote     *raft.RequestVoteArgs        // internal Raft traffic
    AppendEntries   *raft.AppendEntriesArgs
    InstallSnapshot *raft.InstallSnapshotArgs
    TimeoutNow      *raft.TimeoutNowArgs
    ClientRequest   *ClientRequest               // client: Get | Put | Delete | Txn
}
type EnvelopeReply struct {
    /* ... matching reply records ... */
    Err         string
    NotLeader   bool         // "talk to somebody else"
    LeaderHint  string       // address of the current leader, if known
}
```

Direction	Input	Output / meaning
request	one envelope (Raft RPC or client operation)	a fresh TCP connection is opened, the envelope is encoded and sent
reply	—	exactly one `EnvelopeReply`, then the connection closes
redirect	reply with `NotLeader=true`	caller retries automatically at `LeaderHint`
error	—	`Err` carries a human-readable reason

Table 4: Wire Envelope — the single defined way for *any* two nodes (or a client and a node) to talk.

Interface 3: `StatsProvider` (Web Layer → Operational AI)

```
// internal/ai/engine.go
type StatsProvider interface {          // implemented by an adapter in internal/httpapi
    MetricsSnapshot()  metrics.Snapshot
    NodeObservations() []NodeObservation
    VersionCount()     int
```

```
}
```

Method	In	Output	Purpose
MetricsSnapshot	—	current counter/-gauge/histogram values	anomaly detector's raw material
NodeObservations	—	per-node latency, errors, replication lag, queue depth	leadership advisor's raw material
VersionCount	—	number of live MVCC versions	auto-tuner's GC pressure signal

Table 5: `StatsProvider` — everything the AI module is *allowed* to know about the cluster.

Interface 4: Key Routing (Sharding Server → Routing Ring)

```
// internal/shard/ring.go
func NewRing(shards []ID) *Ring
func (r *Ring) ShardFor(key string) ID   // SHA-256 rendezvous (highest-random-weight)
```

Call	Input	Output	Property
ShardFor	`key` (string)	shard `ID` owning that key	deterministic: same key + same shard set ⇒ same answer on any machine, after any restart

Table 6: Key routing — the gateway's only decision mechanism; transactions spanning several shards are rejected with a clear error instead of being applied non-atomically.

8 Step 3 — Cohesion: Two Decisions

Cohesion in one line

Cohesion asks: how closely do the tasks inside *one* module belong together? High cohesion = one module, one main purpose.

Decision C1 — the Write-Ahead Log is a purely generic append-only log (*very high* functional cohesion). `internal/wal` knows nothing about keys, values, transactions or Raft. It accepts arbitrary byte records, appends them, checksums them, fsyncs, and replays them exactly. It has exactly one reason to change. *Low-cohesion equivalent, rejected:* if the WAL also parsed Raft command entries and GC snapshots, it would change for two unrelated reasons (log mechanics *and* consensus semantics) and could not be unit-tested on its own.

Decision C2 — the Operational AI module only analyzes statistics (*very high* single-purpose cohesion). `internal/ai` contains the anomaly detector, the leadership advisor, the auto-tuner, and the natural-language query engine — but not one line that talks to the network, the log, or the storage engine. Its source files were peer-reviewed with this rule: *no import of raft, storage or transport.* The benefit is testability: the module is tested entirely without a cluster running, using fake metrics snapshots.

Module	Cohesion	Why
Write-Ahead Log	very high	one job: safely persist and replay opaque records
Hybrid Clock	very high	one job: unique, ordered timestamps
Version Store	high	one idea: version chains + snapshot isolation + conflict checks
Operational AI	high	one idea: transparent statistics over metric data
Web Layer	medium	several user-facing jobs (API, dashboard, SSE), but all in the same "talk to the outside" role

Table 7: Cohesion of the main modules.

9 Step 4 — Coupling: Two Decisions

Coupling in one line

Coupling asks: how much does *one* module depend on other modules? Low coupling = connections are necessary, limited, clear — ideally through an interface, never by touching another module's private data.

Decision D1 — the Operational AI depends on an interface, not on the cluster (*very low* coupling; interface/dependency-inversion). Everything `internal/ai` sees is the eight lines of `StatsProvider` (Interface 3). The Web Layer supplies a concrete adapter. Consequence: the AI layer can be developed, tested, and even replaced without a cluster, and the cluster can change its internals freely as long as it keeps answering the three methods. *High-coupling equivalent, rejected:* if `internal/ai` imported `internal/raft` directly, every Raft refactor would ripple into the AI code and its tests.

Decision D2 — Consensus sees opaque operations only (*very low* data coupling). `internal/raft` is a state-machine owner, not a database: it replicates and applies *opaque byte operations in a safe order*, and the Transaction Engine interprets them. Consensus therefore never imports the version store's data shapes (beyond a minimal apply callback), which is exactly why the same `internal/raft` could, in principle, replicate a different engine unchanged. *High-coupling equivalent, rejected:* letting Raft know that entry n "is a Put of key k" would couple the hardest-to-test package in the system to the easiest-to-replace part of it.

Controlled exception — the composition root. `internal/httpapi` depends on seven modules (edges 4, 5, 6). This looks like high coupling, but a *single* place must wire the system together and someone must implement the AI adapter; by confining wide dependencies to exactly this module, coupling stays bounded and the other twelve modules remain leaf-like.

Connection	Coupling	Why it is acceptable
Operational AI ↔ cluster	very low	only the StatsProvider interface crosses the boundary
Consensus → state machine	very low	opaque operations; no key/value knowledge leaks in
Version Store → Hybrid Clock	low	a single, tiny, stable data type (the timestamp)
Wire Protocol → Consensus	low	envelope carries Raft records as data, no domain logic exchanged
Web Layer → (many)	medium	deliberate composition root, single point of wiring

Table 8: Coupling of the main connections.

10 Tutorial Numericals

Per the tutorial method: count connections carefully (no double counting), use the given formulas, and always write *formula → substitute → answer → meaning.*

Numerical 1: Count the module connections

Count every arrow of the diagram exactly once (each module pair is connected by exactly one arrow, so no arrow is counted twice):

#	From module	To module	Meaning of the connection
1	CLI Client	Client Library	the CLI speaks through the standard client
2	Sharding Server	Client Library	the gateway proxies reads/writes as a client
3	Sharding Server	Routing Ring	which shard owns this key?
4	Web Layer	Transaction Engine	serve REST reads/writes/txns
5	Web Layer	Operational AI	adapter implements StatsProvider
6	Web Layer	Metrics Registry	export Prometheus / JSON
7	Transaction Engine	Consensus	propose + apply operations
8	Transaction Engine	Version Store	versioned reads/writes, conflicts
9	Transaction Engine	Hybrid Clock	snapshot and commit timestamps
10	Version Store	Hybrid Clock	version ordering by timestamp
11	Consensus	Wire Protocol	send Raft RPCs
12	Consensus	Write-Ahead Log	persist terms, votes, entries, snapshots
13	Client Library	Wire Protocol	encode/sent envelopes
14	Wire Protocol	Metrics Registry	record every request's latency and outcome

$$\text{Total connections} = 14 \qquad \text{Modules drawn} = 13 \qquad \text{Average coupling} = \frac{14}{13} \approx 1.08$$

Meaning: on average each module links to about one other module — the graph is sparse and layered, which is exactly what "necessary, limited and clear" connections look like in practice.

Numerical 2: Stability / instability of a module

Given (standard module-strength formula): a module's instability is

$$I = \frac{F_{\text{out}}}{F_{\text{in}} + F_{\text{out}}},$$

where F_{in} = number of modules depending *on* it and F_{out} = number of modules it depends *on*. ($I = 0$: maximally stable primitive; $I = 1$: maximally volatile top layer.)

Worked example — Wire Protocol: $F_{\text{in}} = 2$ (Consensus 11, Client Library 13); $F_{\text{out}} = 1$ (Metrics Registry 14).

$$I = \frac{1}{2+1} = \frac{1}{3} \approx 0.33$$

Meaning: the Wire Protocol is somewhat stable — two modules lean on its format, so the envelope should only ever be *extended*, never reshaped.

Worked example — Transaction Engine: $F_{\text{in}} = 1$ (Web Layer 4); $F_{\text{out}} = 3$ (Consensus 7, Version Store 8, Hybrid Clock 9).

$$I = \frac{3}{1+3} = 0.75$$

Meaning: the engine is volatile in the good sense: nothing below depends on it, so its transaction API may evolve with little fallout — the opposite direction of what high coupling would give.

Full table (from the fourteen connections above):

Module	F_{in}	F_{out}	I	Reading
CLI Client	0	1	1.00	most volatile: top of the stack
Sharding Server	0	2	1.00	top of the stack
Web Layer	0	3	1.00	composition root by design
Transaction Engine	1	3	0.75	high-level orchestrator
Consensus	1	2	0.67	mid-layer
Version Store	1	1	0.50	balanced
Client Library	2	1	0.33	fairly stable
Wire Protocol	2	1	0.33	fairly stable
Hybrid Clock	2	0	0.00	maximally stable primitive
Metrics Registry	2	0	0.00	maximally stable primitive
Write-Ahead Log	1	0	0.00	maximally stable primitive
Operational AI	1	0	0.00	stable (server-side leaf)
Routing Ring	1	0	0.00	stable (server-side leaf)

Table 9: Instability of all thirteen drawn modules.

Interpretation. I decreases monotonically from Layer 1 to Layer 4 — front ends change freely, primitives change almost never. That one-sentence gradient is the quantitative form of "low coupling, high cohesion": the most complex, most carefully-tested code (Consensus, WAL, HLC) sits where $I \approx 0$, guarded by stable shapes, while the code that must absorb user-facing change sits at $I \approx 1$ with nothing depending on it.

11 Result

Result of the exercise

`kvstore` decomposes cleanly into **13 modules in 4 layers**, with 14 production dependencies between them (average coupling 1.08). Every module has exactly one main responsibility (highest cohesion), lower layers never import upper layers, and the two most powerful modules — Operational AI and Consensus — are detached from the rest through tiny, stable interfaces (StatsProvider, state-machine callbacks), so each can be tested and changed alone. The instability gradient $I : 1.00 \to 0.00$ from front end to primitives confirms quantitatively that the design achieves high cohesion and low coupling.

A Submission Checklist

✓	Checklist item	Where it is satisfied
☒	Module list matches the own project's requirements	Table of requirements R1–R8 → module table
☒	Each module has one main responsibility	"One main responsibility" column of the module table
☒	Diagram has clear names and labeled arrows	Module diagram, edges (1)–(14)
☒	Interfaces specify inputs and outputs	Interfaces 1–4 with signature listings and tables
☒	Cohesion/coupling decisions can be justified	Decisions C1–C2, D1–D2 with rejected alternatives
☒	Numericals: formula → substitute → answer → meaning	Numericals 1–2
☒	Result written in 2–3 lines	Result box

Table 10: Tutorial submission checklist, item by item.

B Lab Time Plan Followed

Time	Phase	What was done
0–10 min	recap & instructions	re-read the tutorial slides' rules and formulas
10–25 min	instructor's example	followed the Library Management System example (Authentication, Catalogue, Loans, …)
25–65 min	own design	Requirements R1–R8, responsibility list, module grouping, module table
65–90 min	interfaces & justification	Interfaces 1–4; cohesion decisions C1–C2; coupling decisions D1–D2
90–100 min	self/peer review	checked diagram names/arrows against the real Go source tree
100–105 min	result & submit	the Result box + this document

Table 11: The 1h 45min lab plan, as executed.

C Module-to-Code Traceability

Diagram node	Code location	Verified by
Transaction Engine	`internal/storage/engine.go`	`Begin/Get/Put/Delete/Rollback/Commit` exist in source
Operational AI	`internal/ai/engine.go`	`StatsProvider` interface declared there
Routing Ring	`internal/shard/ring.go`	`ShardFor` with SHA-256 rendezvous scoring
Wire Protocol	`internal/transport/protocol.go`	`Envelope`, five kinds, `NotLeader`/`LeaderHint`
Consensus	`internal/raft/`	state-machine apply interface + transfer/`TimeoutNow`
Web Layer	`internal/httpapi/`	adapter implements StatsProvider; REST + SSE + dashboard

Table 12: The design is not invented for the lab — every node maps to real, checked code.